

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №3

По дисциплине "Основы машинного обучения" на тему
"Ассоциативные правила"

Группа: Р3324

Выполнил: Пахомов Владислав
Владимирович

Проверила:

к.т.н. доцент Солодка М.А.

Санкт-Петербург

2026

ОГЛАВЛЕНИЕ

Оглавление будет сформировано автоматически при обновлении поля (ПКМ → «Обновить поле»)

ВВЕДЕНИЕ

Цель работы: освоить алгоритмы поиска ассоциативных правил на наборе транзакций розничной торговли. Реализовать с нуля Apriori и FP-Growth, сравнить их по времени работы и построить рекомендательную функцию.

Датасет: Online Retail (UCI 352). После очистки и группировки по номерам счетов получается набор из 20 136 транзакций с 4077 уникальными товарами, средний размер корзины 25,8 позиции.

Задачи работы:

1. Реализовать Apriori с генерацией кандидатов уровень за уровнем и прунингом по антимонотонности.
2. Реализовать FP-Growth через построение FP-дерева и рекурсивный обход условных паттернов.
3. Сравнить время работы алгоритмов при одинаковом `min_support`.
4. Сгенерировать ассоциативные правила по `support`, `confidence` и `lift`.
5. Проверить чувствительность к `min_support` по широкой сетке, затем выполнить детальное сравнение трёх соседних порогов на границе разумного диапазона и обосновать выбор.
6. Реализовать функцию `recommend(basket)`, возвращающую топ рекомендаций по `lift`.

ВЫПОЛНЕНИЕ РАБОТЫ

Описание датасета и предобработка

В исходной таблице 541 909 строк, каждая строка, отдельная позиция в счёте: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country.

Применены три фильтра: убраны строки с InvoiceNo, начинающимся на 'C' (возвраты и отмены), убраны строки с пустым Description, убраны строки с Quantity не больше нуля. После фильтрации остаётся 530 693 строки, удалено 11 216 записей.

Транзакция формируется как множество описаний товаров в рамках одного InvoiceNo. Дубликаты внутри одной транзакции схлопываются, поскольку Apriori работает с наборами, не с мультимножествами. Готовый набор: 20 136 транзакций, 4077 уникальных товаров, средняя корзина 25,8 позиций.

Apriori

Apriori итеративно строит уровни частых наборов L1, L2, L3 и так далее. Поддержка набора X считается как доля транзакций, содержащих X целиком:

$$sup(X) = \frac{|\{T \in D : X \subseteq T\}|}{|D|} \quad (1)$$

где D множество транзакций, T одна транзакция, X набор товаров.

Набор по (1) считается частым, если $sup(X)$ не меньше $min_support$. Ключевое свойство Apriori, антимонотонность поддержки:

$$X \subseteq Y \implies sup(X) \geq sup(Y) \quad (2)$$

Антимонотонность (2) даёт прунинг: если X не частый, ни одно его надмножество не частое, проверять их не нужно. Свойство отсекает кандидатов целыми ветвями.

Шаги алгоритма:

1. Подсчитать support одиночных товаров, оставить с support не меньше min_support , получить L_1 .
2. Сгенерировать кандидатов $C(k+1)$ объединением пар из L_k : каждое объединение длины $k+1$ берётся не больше одного раза.
3. Прунинг: оставить только тех кандидатов, у которых все k -подмножества входят в L_k .
4. Один проход по всем транзакциям: посчитать support оставшихся кандидатов.
5. Кандидаты с support не меньше min_support формируют $L(k+1)$. Если пусто, остановиться.

При $\text{min_support}=0,03$ алгоритм находит 136 частых наборов, из которых 128 одиночных и 8 пар. Время работы 4,96 секунды на 20 136 транзакциях.

FP-Growth

FP-Growth работает иначе: вместо генерации кандидатов он сжимает все транзакции в одну компактную структуру (FP-дерево) и рекурсивно строит условные деревья для каждого товара. Преимущество: транзакции сканируются всего дважды, не на каждом уровне k .

Алгоритм:

1. Первый проход: подсчитать частоты одиночных товаров, оставить только частые.
2. Второй проход: для каждой транзакции отсортировать оставшиеся товары по убыванию частоты и вставить в FP-дерево. Каждый узел хранит item, count и ссылку на родителя.
3. Параллельно ведём header-таблицу: для каждого товара связный список всех его узлов в дереве.
4. Майнинг: для каждого товара (от редких к частым) собираем conditional pattern base (пути от корня до его узлов с весами), строим conditional FP-tree и рекурсивно ищем в нём частые наборы с фиксированным префиксом.

FP-Growth при том же $\text{min_support}=0,03$ находит те же 136 наборов с теми же значениями support , проверка $\text{'freq_apriori == freq_fpg'}$ проходит. Время работы 0,38 секунды.

Сравнение производительности

Чтобы оценить разрыв между алгоритмами не на одной точке, а в динамике, прогоняю оба алгоритма на трёх соседних значениях порога: 0,02, 0,025 и 0,03. На каждом значении проверяется равенство результатов ($\text{'freq_apriori == freq_fpg'}$), число частых наборов и время работы. Результат на рисунке 1.

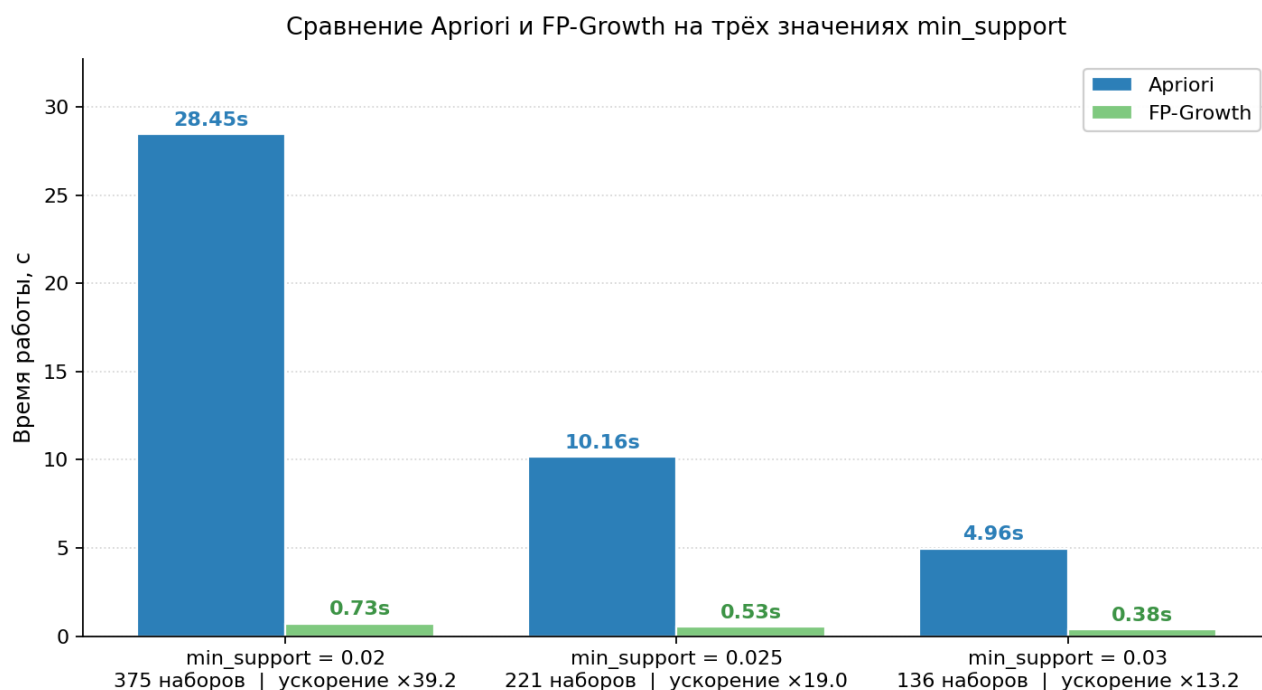


Рисунок 1 — Время работы Apriori и FP-Growth при разных значениях min_support

Таблица 1 — Сравнение Apriori и FP-Growth на 20 136 транзакциях

min_support	Apriori, с	FP-Growth, с	Ускорение	Частых наборов
0,02	28,45	0,73	39×	375
0,025	10,16	0,53	19×	221
0,03	4,96	0,38	13×	136

Поведение по таблице 2: при понижении порога с 0,03 до 0,02 Apriori замедляется почти в шесть раз (с 4,96 до 28,45 секунды), FP-Growth, всего в два (с 0,38 до 0,73). Разрыв расширяется с $13\times$ до $39\times$.

Причина в том, как растут затраты на этап генерации кандидатов. У Apriori на втором уровне число пар-кандидатов примерно равно $|L1| * (|L1| - 1) / 2$, где $|L1|$, число частых одиночных товаров. При 0,03 имеем $|L1| = 128$ и 8128 кандидатов; при 0,02 порог пропускает больше одиночек, и число кандидатов растёт квадратично. Каждый кандидат сравнивается со всеми 20 136 транзакциями, отсюда взрывной рост времени. У FP-Growth кандидатов как таковых нет: все частые наборы извлекаются обходом дерева, а дерево увеличивается медленнее линейного. Поэтому время растёт намного мягче.

Отдельно отмечу, что при 0,02 Apriori успевает дойти до третьего уровня (появляются триплеты), и для них генерация и проверка ещё прибавляются. У FP-Growth обход дерева на этот уровень масштабируется почти бесплатно.

Генерация правил

Из каждого частого набора длиной не меньше 2 порождаются правила вида antecedent $\rightarrow$ consequent с consequent из одного товара. Для пары {A, B} получаются два правила: $A \rightarrow B$ и $B \rightarrow A$. Поддержка совместного появления:

$$sup(A \rightarrow B) = sup(A \cup B) = \frac{|\{T \in D : A \cup B \subseteq T\}|}{|D|} \quad (3)$$

Поддержка (3) показывает долю транзакций, где встречаются одновременно A и B. Достоверность это условная вероятность B при условии A:

$$conf(A \rightarrow B) = \frac{sup(A \cup B)}{sup(A)} \quad (4)$$

Достоверность (4) показывает, как часто срабатывает правило. Подъём сравнивает достоверность с базовой частотой B и снимает смещение в сторону популярных товаров:

$$lift(A \rightarrow B) = \frac{conf(A \rightarrow B)}{sup(B)} = \frac{sup(A \cup B)}{sup(A) \cdot sup(B)} \quad (5)$$

Lift по (5) больше единицы, А и В встречаются вместе чаще случайного. Lift меньше единицы, отрицательная связь, такие правила не интересны для рекомендаций. Финальная фильтрация по двум порогам:

$$sup(A \rightarrow B) \geq \tau_{sup}, \quad conf(A \rightarrow B) \geq \tau_{conf} \quad (6)$$

Критерий (6) с $\tau_{sup} = 0,03$ и $\tau_{conf} = 0,5$ даёт 11 правил, отсортированных по убыванию lift, приведены в таблице 2.

Таблица 2 — Правила, отсортированные по убыванию lift

#	Antecedent	Consequent	support	confidence	lift
1	GREEN REGENCY TEACUP AND SAUCER	PINK REGENCY TEACUP AND SAUCER	0,031	0,62	16,39
2	PINK REGENCY TEACUP AND SAUCER	GREEN REGENCY TEACUP AND SAUCER	0,031	0,83	16,39
3	GREEN REGENCY TEACUP AND SAUCER	ROSES REGENCY TEACUP AND SAUCER	0,038	0,76	14,29
4	ROSES REGENCY TEACUP AND SAUCER	GREEN REGENCY TEACUP AND SAUCER	0,038	0,72	14,29
5	ALARM CLOCK BAKELIKE RED	ALARM CLOCK BAKELIKE GREEN	0,032	0,61	12,51

6	ALARM CLOCK BAKELIK E GREEN	ALARM CLOCK BAKELIK E RED	0,032	0,65	12,51
7	LUNCH BAG PINK POLKADO T	LUNCH BAG RED RETROSP OT	0,030	0,56	7,16
8	JUMBO BAG PINK POLKADO T	JUMBO BAG RED RETROSP OT	0,041	0,68	6,52
9	LUNCH BAG BLACK SKULL	LUNCH BAG RED RETROSP OT	0,032	0,50	6,48
10	JUMBO STORAGE BAG SUKI	JUMBO BAG RED RETROSP OT	0,036	0,61	5,89
11	JUMBO SHOPPER VINTAGE RED PAISLEY	JUMBO BAG RED RETROSP OT	0,034	0,58	5,57

Видно три группы товаров-компаньонов: фарфоровые чашки Regency разных цветов, будильники Bakelike разных цветов, тканевые сумки Lunch/Jumbo разных принтов. Lift в 12-16 говорит о сильной связи внутри коллекций.

Чувствительность к min_support

Чтобы обосновать выбор порога, прогоним FP-Growth по широкой сетке min_support и посмотрим, как меняется число частых наборов и число правил при том же min_confidence = 0,5. Результат на рисунке 2.

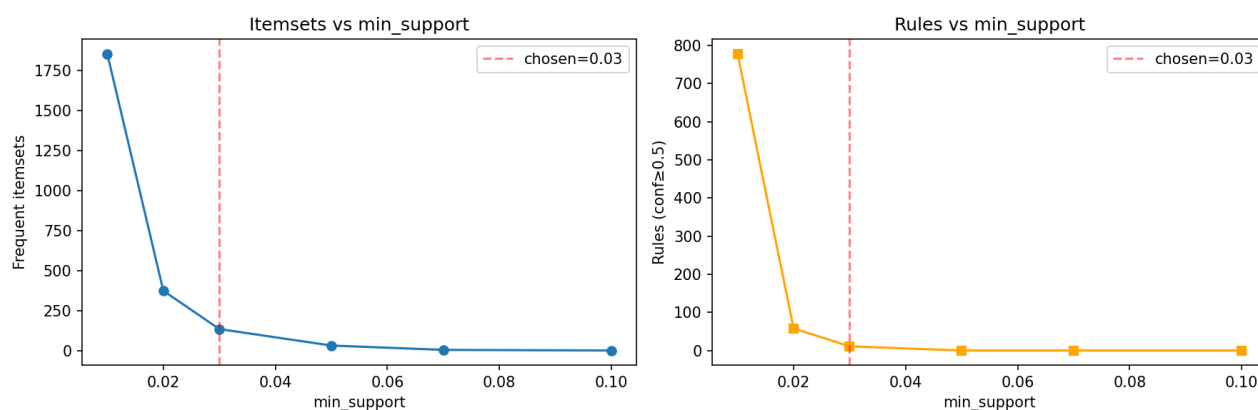


Рисунок 2 — Зависимость числа частых наборов и правил от min_support

Таблица 3 — Чувствительность результата к min_support

min_support	Itemsets	Rules
0,01	1854	778
0,02	375	58
0,03	136	11
0,05	33	0
0,07	6	0
0,10	2	0

Картина по сетке: min_support=0,01 даёт почти 800 правил, многие из них шумные и плохо интерпретируются; при min_support=0,05 и выше теряются интересные паттерны, под порог не проходят даже сильные пары вроде Regency Теасир. Разумный диапазон лежит между 0,02 и 0,03. Чтобы выбрать конкретное значение осознанно, одних агрегированных счётчиков мало, поэтому ниже разберём три соседние точки на этой границе подробнее.

Детальное сравнение порогов 0,02 / 0,025 / 0,03

На широкой сетке между 0,02 и 0,03 переход выглядит плавным, но количество правил меняется с 11 до 58, в пять раз. Чтобы понять, какие правила добавляются и стоят ли они того, прогоним три точки и посмотрим не только на счётчики, но и на структуру: глубину частых наборов (уровень k=3 в Apriori появляется только при определённом пороге) и характер правил, варианты одного семейства продуктов против настоящих кросс-продуктовых связей.

Для классификации правил используем простую эвристику: правило считаем вариантом, если после удаления токенов цвета, паттерна и размера

(PINK, GREEN, RED, POLKADOT, RETROSPOT, BAKELIKE, REGENCY, STORAGE, LARGE, SMALL и так далее) стем антецедента совпадает со стемом консеквента. Например, PINK REGENCY TEACUP AND SAUCER и GREEN REGENCY TEACUP AND SAUCER после страйпа дают одинаковое TEACUP AND SAUCER, значит, перед нами вариант. Остальные правила помечаем как кросс-продуктовые.

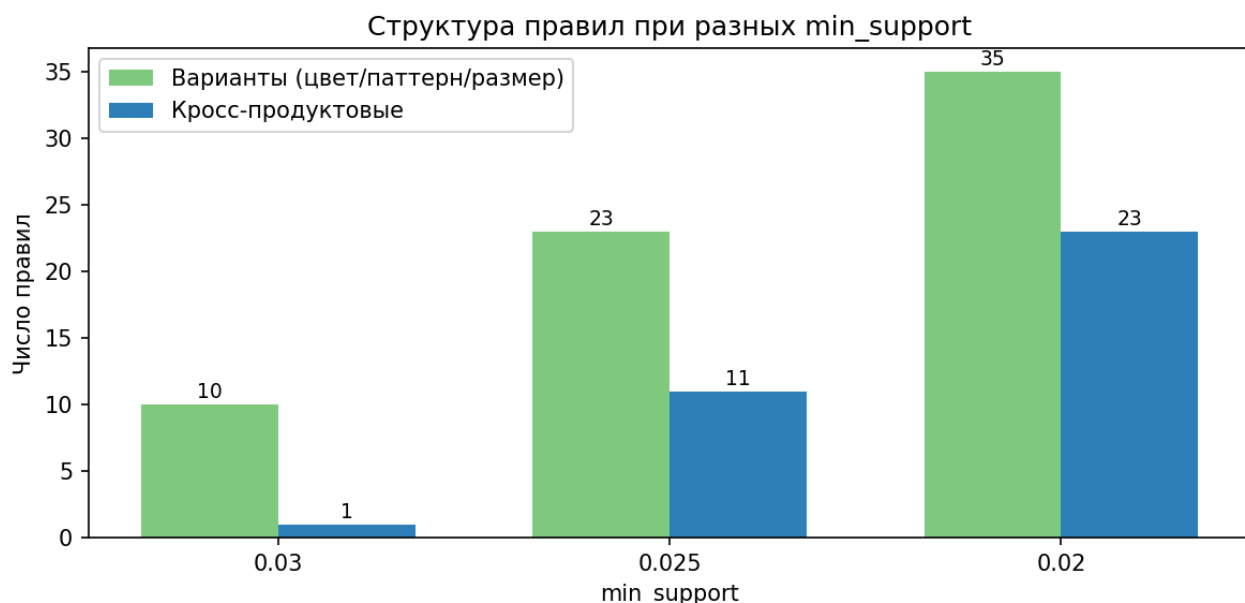


Рисунок 3 — Структура правил при разных min_support: варианты против кросс-продуктовых

Таблица 4 — Структурное сравнение порогов 0,02 / 0,025 / 0,03

min_support	Itemsets	k=1	k=2	k=3	Rules	Variant	Cross
0,03	136	128	8	0	11	10	1
0,025	221	186	34	1	34	23	11
0,02	375	293	79	3	58	35	23

Из таблицы 4 видно три важных эффекта понижения порога. Первый, Apriori начинает работать на нескольких уровнях: при 0,03 ни одного частого тройного набора, при 0,025 появляется один (триплет цветов Regency), при 0,02 уже три триплета, в том числе с участием Cakestand. То есть только при пороге не выше 0,025 Apriori проходит полный цикл генерации кандидатов по уровням, ради которого этот алгоритм вообще нужен.

Второй эффект, доля кросс-продуктовых правил растёт значимо. При 0,03 это всего одно правило из 11; при 0,025 их уже 11 из 34; при 0,02, 23 из 58. То есть основной прирост числа правил при понижении порога приходится именно на связи между разными семействами товаров, а не на дубли вариантов внутри одной коллекции.

Третий эффект, среди новых кросс-продуктовых правил появляются содержательно интересные. При 0,025 это GARDENERS KNEELING PAD CUP OF TEA ↔ KEEP CALM (lift 15,89, разные надписи на одной серии подушек), SPACEBOY LUNCH BOX ↔ DOLLY GIRL LUNCH BOX (lift 14,16, две тематические серии ланч-боксов), PAPER CHAIN KIT VINTAGE CHRISTMAS ↔ 50'S CHRISTMAS (lift 11,69, два стилевых варианта одного праздничного набора). При 0,02 добавляется ключевое кросс-категорийное правило GREEN REGENCY TEACUP AND SAUCER → REGENCY CAKESTAND 3 TIER (lift 5,08), классическое complement-правило чайного сервиза, чашка к подставке для торта, а не очередной вариант цвета чашки. Также появляются связи по размеру (HEART OF WICKER LARGE → SMALL, lift 8,93) и по размеру упаковки (60 TEATIME FAIRY CAKE CASES → PACK OF 72 RETROSPOT CAKE CASES, lift 8,35).

Сравнение производительности при понижении порога тоже играет в пользу FP-Growth: число частых наборов растёт с 136 до 375, число пар-кандидатов в Apriori растёт квадратично, разрыв с FP-Growth расширяется. Получается, выбор 0,02 одновременно делает рекомендации содержательнее и резче подчёркивает преимущество FP-Growth перед Apriori.

С другой стороны, при 0,02 появляются правила-паразиты: антецедент содержит достаточно популярный товар, а консеквент, JUMBO BAG RED RETROSPOT (топ-сумка магазина, встречается в значительной доле корзин). У таких правил высокий lift только за счёт частоты правой части. Чтобы это компенсировать, при переходе на 0,02 имеет смысл поднять min_confidence до 0,55-0,6, тогда большая часть паразитов отсекается, и остаётся ровно тот корпус правил, который нужен для рекомендаций.

Итог: при 0,03 правила самые интерпретируемые и с самым высоким lift, но все они, варианты одного семейства, и Apriori на этом пороге не доходит до $k \geq 3$. При 0,02 за счёт большего числа кандидатов появляются реальные кросс-категорийные связи, и оба алгоритма работают на нескольких уровнях. В разделе рекомендательной функции ниже базовый набор правил берётся при 0,03 как самый консервативный и читабельный вариант.

Рекомендательная функция

Функция `recommend(basket, rules, top_n)` принимает текущую корзину и выдаёт `top_n` рекомендаций. Логика:

1. Перебрать все правила, оставить те, где `antecedent` целиком входит в `basket`.
2. Из `consequent` взять товары, которых ещё нет в корзине.
3. Если один и тот же товар рекомендован несколькими правилами, оставить тот вариант, у которого выше `lift`.
4. Отсортировать кандидатов по убыванию `lift`, вернуть `top_n`.

Сортировка по `lift`, а не по `confidence`, важна: `lift` штрафует популярные товары, которые покупают и без рекомендации. Без этого выдача забила бы топовыми позициями каталога.

Пример работы для одиночной корзины:

```
basket = {'GREEN REGENCY TEACUP AND SAUCER'}
recommend(basket, rules)
-> PINK REGENCY TEACUP AND SAUCER (confidence=0,62, lift=16,4)
-> ROSES REGENCY TEACUP AND SAUCER (confidence=0,76, lift=14,3)
```

И для корзины из двух разнородных товаров:

```
basket = {'GREEN REGENCY TEACUP AND SAUCER', 'JUMBO SHOPPER VINTAGE
RED PAISLEY'}
recommend(basket, rules)
-> PINK REGENCY TEACUP AND SAUCER (confidence=0,62, lift=16,4)
-> ROSES REGENCY TEACUP AND SAUCER (confidence=0,76, lift=14,3)
-> JUMBO BAG RED RETROSPOT (confidence=0,58, lift=5,6)
```

Видно, что для второго товара срабатывает правило про Jumbo-сумки, оно корректно дополняет рекомендацию из первой группы.

ЗАКЛЮЧЕНИЕ

1. Apriori и FP-Growth реализованы с нуля на чистом Python с numpy. Полное совпадение результата (`freq_apriori == freq_fpg`` на 136 наборах с одинаковыми support-счётчиками) подтверждает корректность обеих реализаций.
2. Сравнение производительности проведено на трёх значениях `min_support`: 0,02, 0,025 и 0,03. При 0,03 FP-Growth быстрее Apriori в 13 раз (0,38 с против 4,96 с); при 0,025, в 19 раз (0,53 с против 10,16 с); при 0,02, в 39 раз (0,73 с против 28,45 с). Разрыв расширяется потому, что у Apriori число кандидатов на втором уровне растёт квадратично от числа частых одиночек, и каждый кандидат сверяется со всеми 20 136 транзакциями. FP-Growth кандидатов не генерирует, всё извлекается обходом сжатого дерева.
3. Из 136 частых наборов при пороге 0,03 получено 11 правил с lift от 5,57 до 16,39. Лидируют пары товаров-компаньонов из одной коллекции: чашки Regency разных цветов, будильники Bakelike, тканевые сумки Lunch и Jumbo с разными принтами.
4. Анализ чувствительности к `min_support` по широкой сетке выделил диапазон 0,02-0,03 как разумный; крайние значения 0,01 и 0,05 дают либо шум (1854 наборов, 778 правил), либо пустоту (ноль правил).
5. Детальное сравнение трёх соседних порогов 0,02 / 0,025 / 0,03 показало качественное различие, не видимое по агрегированным счётчикам. Только при пороге не выше 0,025 Apriori доходит до уровня $k \geq 3$ (триплеты появляются), а при 0,02 число триплетов растёт до трёх. Доля кросс-продуктовых правил с понижением порога растёт с 1 из 11 (при 0,03) до 23 из 58 (при 0,02). При 0,02 появляются содержательные кросс-категорийные связи: GREEN REGENCY TEACUP $\rightarrow$ REGENCY CAKESTAND 3 TIER (чашка к подставке для торта), GARDENERS KNEELING PAD CUP OF TEA $\leftrightarrow$ KEEP CALM, SPACEBOY $\leftrightarrow$ DOLLY GIRL LUNCH BOX, размерные связи HEART OF WICKER LARGE $\rightarrow$ SMALL. Порог 0,03 даёт самые

консервативные и читабельные правила, 0,02 расширяет каталог и сильнее задействует оба алгоритма по уровням, что также увеличивает разрыв во времени между Apriori и FP-Growth.

6. Рекомендательная функция `recommend(basket)` ранжирует кандидатов по lift, что не даёт популярным товарам забить рекомендации. Корректно работает и с одним товаром в корзине, и с несколькими товарами из разных категорий.